Network Intrusion Detection Using Machine Learning

A Mini Project Report

Submitted by: Atchayasree Department of Computer Science and Engineering KIT – Kalaignarkarunanidhi Institute of Technology, Coimbatore Academic Year: 2026–2027
Project Guide: [Guide name]

Abstract

As organisations depend more on networked systems, the volume and sophistication of cyber attacks has grown sharply. Traditional signature-based security tools can only detect attacks they already know about, and they struggle with zero-day and slightly modified attacks. This project presents a Network Intrusion Detection System (NIDS) that uses machine learning to classify network traffic as either normal or malicious. The system is trained on the NSL-KDD benchmark dataset, which contains labelled network connection records covering Denial of Service (DoS), Probe, Remote-to-Local (R2L) and User-to-Root (U2R) attacks. The pipeline covers data cleaning, categorical encoding, feature scaling, feature selection and model training using Random Forest, Decision Tree, K-Nearest Neighbours and Support Vector Machine classifiers. Models are compared on accuracy, precision, recall, F1-score and false alarm rate. The best model, Random Forest, achieves high detection accuracy with a low false positive rate, showing that machine learning is a practical complement to rule-based intrusion detection.

Keywords: Network security, intrusion detection, machine learning, NSL-KDD, Random Forest, anomaly detection

Table of Contents

1. Introduction
2. Problem Statement and Objectives
3. Literature Review
4. System Requirements
5. Dataset Description
6. Proposed System and Methodology

7. Implementation
8. Results and Discussion
9. Limitations
10. Conclusion and Future Scope
11. References

1. Introduction

Computer networks carry sensitive information such as financial records, personal data and business communication. Every device connected to a network is a possible entry point for an attacker. Reports from security vendors consistently show that the number of attacks, and the damage they cause, increases every year. Firewalls and antivirus software provide a first layer of defence, but they cannot inspect and reason about all the behaviour happening inside a network. An Intrusion Detection System (IDS) fills this gap by monitoring traffic or system activity and raising an alert when something suspicious occurs.

An IDS can be classified in two ways. By deployment, a **Host-based IDS (HIDS)** monitors a single machine's logs and files, whereas a **Network-based IDS (NIDS)** monitors packets and flows travelling across a network segment. By detection method, a **signature-based IDS** compares traffic with a database of known attack patterns, whereas an **anomaly-based IDS** learns what normal behaviour looks like and flags deviations from it.

Signature-based systems are accurate on known attacks but are blind to new ones, and their rule sets need constant manual updates. Anomaly-based systems can detect new attack types but historically suffered from high false alarm rates. Machine learning offers a middle path: a model can learn the statistical patterns that separate attack traffic from normal traffic directly from labelled data, and generalise to variants it has not seen exactly before.

This mini project builds a machine-learning-based NIDS and evaluates how well different classifiers perform on this task.

2. Problem Statement and Objectives

2.1 Problem Statement

To design and implement a machine-learning-based Network Intrusion Detection System that analyses network connection records and accurately classifies each record as normal or as one of the major attack categories, while keeping the false alarm rate low.

2.2 Objectives

- To study the types of network attacks and the working of intrusion detection systems.
- To preprocess the NSL-KDD dataset, including encoding of categorical features and scaling of numerical features.
- To reduce dimensionality by selecting the most informative features.
- To train and compare multiple supervised classifiers for binary (normal vs attack) and multi-class (normal, DoS, Probe, R2L, U2R) detection.
- To evaluate the models using standard metrics and identify the best performing approach.

2.3 Scope

The project works on pre-captured, labelled connection records and operates offline. Live packet capture and real-time deployment are outside the scope of this mini project and are discussed under future work.

3. Literature Review

Early work on intrusion detection by Denning introduced the idea that intrusions can be found by identifying abnormal usage patterns. The NIST guide to intrusion detection and prevention systems (SP 800-94) later formalised the categories of IDS technologies and their deployment considerations.

The KDD Cup 1999 dataset became the most widely used benchmark for evaluating learning-based intrusion detectors. Tavallaee et al. analysed this dataset and found serious problems: a large share of duplicate records that biased classifiers towards frequent samples, and test records that were too easy. They proposed NSL-KDD, a cleaned version without redundant records, which is now a standard benchmark and is used in this project.

On the algorithm side, ensemble methods such as Random Forest, introduced by Breiman, have repeatedly performed well on tabular intrusion data because they handle mixed feature types, resist overfitting and give feature importance scores. Boosting methods such as XGBoost have also reported strong results. More recent studies use deep learning models, such as autoencoders, CNNs and LSTMs, particularly on larger and more modern datasets such as CICIDS2017 by Sharafaldin et al., which reflect current traffic and attack types.

The literature shows that no single method is best in every setting. Classical machine learning models are fast, easy to explain and perform well on structured features, which makes them a sensible choice for a mini project and a good baseline for later deep learning work.

4. System Requirements

4.1 Hardware

- Processor: Intel Core i5 or equivalent
- RAM: 8 GB minimum
- Storage: 2 GB free space

4.2 Software

- Operating system: Windows 10/11 or Linux
- Language: Python 3.10 or above
- Libraries: NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn
- Environment: Jupyter Notebook or Google Colab

5. Dataset Description

The **NSL-KDD** dataset was used. It is provided as two files, KDDTrain+ (about 125,973 records) and KDDTest+ (about 22,544 records). Each record describes a single network connection with **41 features** plus a label and a difficulty score. The features fall into four groups:

Feature group	Description	Examples
Basic features	Properties of the connection itself	duration, protocol_type, service, flag, src_bytes, dst_bytes
Content features	Behaviour inside the connection payload	num_failed_logins, logged_in, root_shell, num_file_creations
Time-based traffic features	Statistics over the last 2 seconds	count, srv_count, serror_rate, rerror_rate
Host-based traffic features	Statistics over the last 100 connections to a host	dst_host_count, dst_host_srv_count, dst_host_same_srv_rate

The attack records are grouped into four main categories:

Category	Meaning	Examples
DoS	Denial of Service: overwhelm a target so it becomes unavailable	neptune, smurf, back, teardrop
Probe	Scanning a network to gather information	nmap, satan, portsweep, ipsweep
R2L	Remote to Local: unauthorised access from a remote machine	guess_passwd, ftp_write, warezclient
U2R	User to Root: a normal user gains administrator privileges	buffer_overflow, rootkit, loadmodule

The dataset is imbalanced: DoS and normal records dominate, while U2R and R2L are rare. This affects how the results should be interpreted, especially recall on the minority classes.

6. Proposed System and Methodology

6.1 System Architecture

The system follows a standard machine learning pipeline:

Network traffic records → Data preprocessing → Feature selection → Model training → Classification → Alert (Normal / Attack type)

The dataset is split into a training portion for learning and a testing portion for evaluation. The trained model then predicts a class for each unseen connection record.

6.2 Data Preprocessing

1. **Loading and labelling:** column names were assigned and the attack label was mapped to its category (Normal, DoS, Probe, R2L, U2R). For binary classification, every attack label was mapped to a single class, Attack.
2. **Cleaning:** missing values and duplicate rows were checked and removed.
3. **Encoding:** the categorical features protocol_type, service and flag were converted to numbers using one-hot encoding.
4. **Scaling:** numerical features such as src_bytes and dst_bytes have very different ranges, so they were normalised using Min-Max scaling, which is especially important for distance-based models such as KNN and SVM.

6.3 Feature Selection

Not all 41 features contribute equally. Using all of them increases training time and may add noise. Feature importance scores from a Random Forest, together with correlation analysis, were used to keep the most informative features. Features such as src_bytes, dst_bytes, count, srv_count, same_srv_rate, dst_host_srv_count and serror_rate consistently ranked high, which is consistent with how DoS and Probe attacks generate large numbers of similar connections.

6.4 Classification Algorithms

Decision Tree: builds a tree of if-else rules on feature values. It is fast and easy to interpret, but a single tree can overfit.

Random Forest: trains many decision trees on random subsets of data and features, and combines their votes. This reduces variance and typically gives much better generalisation than a single tree.

K-Nearest Neighbours (KNN): classifies a record by the majority label among its k closest records in feature space. It needs no training phase but is slow at prediction time on large datasets.

Support Vector Machine (SVM): finds the hyperplane that best separates the classes with the maximum margin, and can use kernels for non-linear boundaries. It works well on scaled data but trains slowly on large datasets.

6.5 Evaluation Metrics

Let TP, TN, FP and FN be the true positives, true negatives, false positives and false negatives, where "positive" means an attack.

- **Accuracy** = (TP + TN) / (TP + TN + FP + FN)
- **Precision** = TP / (TP + FP)
- **Recall (Detection Rate)** = TP / (TP + FN)
- **F1-score** = 2 × Precision × Recall / (Precision + Recall)
- **False Alarm Rate** = FP / (FP + TN)

In intrusion detection, recall matters because a missed attack is costly, while a low false alarm rate matters because too many false alerts cause analysts to ignore the system. Accuracy alone can be misleading on imbalanced data, so all metrics are reported.

7. Implementation

The project was implemented in Python using Scikit-learn. The main steps are shown below.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# 1. Load data
df = pd.read_csv("KDDTrain+.txt", names=columns)
df["label"] = df["attack"].apply(lambda x: 0 if x == "normal" else 1)

# 2. Encode categorical features
df = pd.get_dummies(df, columns=["protocol_type", "service", "flag"])

# 3. Split and scale
X = df.drop(["attack", "label", "difficulty"], axis=1)
y = df["label"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
scaler = MinMaxScaler().fit(X_train)
X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)
```

```
# 4. Train and evaluate
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print(classification_report(y_test, pred))
print(confusion_matrix(y_test, pred))
```

The scaler is fitted only on the training data and then applied to the test data, so that no information from the test set leaks into training. The same pipeline was repeated for the Decision Tree, KNN and SVM classifiers, and for the multi-class labels. Hyperparameters such as the number of trees, tree depth and the value of k were tuned using cross-validation.

8. Results and Discussion

Note: the values below are indicative of what is typically observed for these models on NSL-KDD with an 80/20 split of KDDTrain+. Replace them with the exact numbers from your own run before submission.

8.1 Binary Classification (Normal vs Attack)

Model	Accuracy	Precision	Recall	F1-score	False Alarm Rate
Decision Tree	99.5%	99.5%	99.5%	99.5%	0.5%
Random Forest	99.8%	99.8%	99.8%	99.8%	0.2%
KNN (k = 5)	99.2%	99.2%	99.3%	99.2%	0.9%
SVM (RBF kernel)	98.7%	98.6%	98.9%	98.7%	1.4%

Random Forest gave the best overall balance of detection rate and false alarm rate. Decision Tree came a close second and trains fastest. KNN performed well but was noticeably slower during prediction. SVM was the slowest to train and slightly less accurate.

8.2 Multi-class Classification

For five-class classification, DoS and Probe were detected with very high recall because their traffic patterns are distinctive and well represented. R2L and especially U2R had much lower recall. They have few training samples and their traffic closely resembles

normal user behaviour. This is the well known weakness of NSL-KDD based detectors and is visible in the confusion matrix, where U2R records are frequently mislabelled as Normal.

8.3 Evaluation on the Official Test Set

When the same models are tested on KDDTest+, the official test file, accuracy is considerably lower than on a random split of the training file, typically in the range of 75–82%. This is because KDDTest+ contains attack types that do not appear in the training data. The gap shows that high accuracy on a random split overstates how well a model handles genuinely new attacks, and it highlights the importance of testing on data that differs from the training distribution.

8.4 Discussion

- Ensemble learning gave the most reliable results, which agrees with the findings reported in the literature.
- Feature selection reduced training time noticeably with almost no loss in accuracy.
- Class imbalance, not model choice, was the main reason for weak performance on U2R and R2L.
- Scaling was essential for KNN and SVM but made little difference to tree-based models.

9. Limitations

- NSL-KDD is derived from 1998 traffic. Modern protocols, encrypted traffic and current attack techniques are not represented.
- The system analyses stored records offline and does not capture live packets.
- The minority classes (R2L and U2R) are detected poorly.
- Supervised models detect only the kinds of attacks present in the training data and need retraining as threats evolve.

10. Conclusion and Future Scope

10.1 Conclusion

This project implemented a machine-learning-based Network Intrusion Detection System on the NSL-KDD dataset. After cleaning, encoding, scaling and selecting features, four classifiers were trained and compared. Random Forest produced the best results, with very high accuracy and a low false alarm rate in binary classification. The experiments also showed that class imbalance and the gap between training and unseen attack types remain the main practical challenges. Overall, the work demonstrates that machine learning can significantly improve intrusion detection over purely rule-based methods and can be used alongside existing tools such as firewalls and signature-based systems.

10.2 Future Scope

- Train and test on newer datasets such as CICIDS2017 or UNSW-NB15 that reflect modern traffic.
- Handle class imbalance with SMOTE oversampling or cost-sensitive learning to improve R2L and U2R detection.
- Use deep learning models such as autoencoders and LSTMs for anomaly detection and sequence modelling.
- Integrate live packet capture using tools such as Scapy or Zeek and build a real-time detection and alerting dashboard.
- Deploy a lightweight version of the model on edge devices or routers.

11. References

1. M. Tavallaee, E. Bagheri, W. Lu and A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," IEEE Symposium on Computational Intelligence for Security and Defense Applications, 2009.
2. K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST Special Publication 800-94, 2007.
3. L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
4. I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," ICISSP, 2018.
5. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

6. T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," ACM SIGKDD, 2016.
7. D. E. Denning, "An Intrusion-Detection Model," IEEE Transactions on Software Engineering, vol. SE-13, no. 2, 1987.